

SRE Hands-On Internal Evaluation

Notes API on k3s

Enterprise Implementation Report

Deployment · CI/CD · Observability · Resilience · Incident Response

Prepared by: Vishnu K

Prepared for: Nerdience Technologies

Date: August 2026

Repository: github.com/vishnucax/SRE-Project1

Live environment: <http://13.126.63.217>

Table of Contents

1. Executive Summary	4
2. Project Overview	5
2.1 Objective.....	5
2.2 Scope.....	5
2.3 Deliverables.....	5
2.4 Technology stack	5
3. Solution Architecture	6
3.1 Component explanations.....	6
4. Infrastructure Design	7
4.1 Compute & operating system	7
4.2 Kubernetes (k3s) & networking	7
4.3 Configuration, secrets & storage.....	7
5. Application Deployment	8
5.1 The application	8
5.2 Container image	8
5.3 Deployment flow	8
6. CI/CD Pipeline.....	9
6.1 Key design points.....	10
7. Monitoring & Observability.....	10
7.1 Metrics & scraping.....	10
7.2 Alerting rules	10
7.3 Recording rules.....	10
8. Grafana Dashboards	11
8.1 Service-health dashboard.....	11
8.2 Cluster / infrastructure dashboard	11
9. Logging Architecture	12
10. Failure Simulations	12
10.1 Simulation 1 — Pod Crash Loop (configuration failure).....	12
10.2 Simulation 2 — Database Connectivity Loss (dependency failure)	14
10.3 Simulation 3 — High CPU Exhaustion (resource failure)	15
11. Incident Response (Postmortems).....	16
11.1 Incident #1 — Pod Crash Loop	16
11.2 Incident #2 — Database Connectivity Loss	17
11.3 Incident #3 — High CPU Exhaustion	17
12. Runbooks.....	18
12.1 First five minutes (triage).....	18
12.2 Pod crash loop.....	18
12.3 Database connectivity loss	18
12.4 High CPU	18
13. Security.....	19

13.1 Secrets handling.....	19
13.2 Network segmentation caveat.....	19
13.3 Observed exploit scanning	19
14. Project Folder Structure	20
15. Challenges Faced	20
16. Lessons Learned	20
17. Future Improvements	20
18. Conclusion.....	21
Appendix A — Key Commands	21
Appendix B — Configuration Files.....	21
Appendix C — References	21
Appendix D — Abbreviations	21
Appendix E — Glossary.....	22

1. Executive Summary

The problem

Modern services must be deployed repeatably, observed continuously, and recovered quickly when they fail. This evaluation asked for a working demonstration of those Site Reliability Engineering (SRE) practices on a single Linux virtual machine, end to end.

The solution

A small Flask 'Notes API' microservice, backed by PostgreSQL, was deployed on a single-node k3s Kubernetes cluster on AWS. Every code change is automatically built, security-scanned, published, and deployed by a GitHub Actions pipeline. The running service is monitored with Prometheus, visualised in Grafana, and its logs aggregated in Loki. Three failure scenarios were deliberately induced, detected by alerts, documented with root-cause analyses, and recovered.

Technologies

k3s (Kubernetes), Docker, Helm, GitHub Actions, ghcr.io, Prometheus, Grafana, Loki, Promtail, PostgreSQL, Flask, and Trivy.

The result

- A live, reachable application deployed through a fully automated pipeline.
- Complete observability: metrics, dashboards, logs, five alert rules and three recording rules.
- Three documented incidents with blameless postmortems and an operational runbook.
- A security posture with secrets handling, RBAC, image scanning, and honestly documented limitations.

In short: the service is not just deployed — it is observable, testable, and operable, which is the core of SRE.

2. Project Overview

2.1 Objective

Demonstrate practical SRE skills across provisioning, containerization, orchestration, CI/CD automation, monitoring and alerting, logging, resilience engineering (failure simulation), and incident response (RCA and runbook creation).

2.2 Scope

The application is intentionally simple — it is a vehicle for demonstrating operational practices, not an exercise in application complexity. The work is scoped to a single-node cluster; limitations arising from that (for example, the database being a single point of failure) are documented honestly rather than hidden.

2.3 Deliverables

- Application source, Dockerfile, and Helm chart.
- GitHub Actions CI/CD workflow (build, scan, push, deploy).
- Prometheus configuration (scrape config, alerting rules, recording rules), Grafana dashboards (JSON exports + import instructions), and Loki/Promtail configuration.
- Three failure-simulation scripts, three RCAs, an operational runbook, security notes, and design notes.
- A README, this report, a summary document, a presentation, and a demo video.

2.4 Technology stack

Layer	Technology	Purpose
Cloud / VM	AWS EC2 (Ubuntu 24.04, t3.xlarge)	Host environment
Orchestration	k3s	Lightweight Kubernetes; Traefik ingress built in
Application	Python Flask + gunicorn	The microservice
Database	PostgreSQL (in-cluster, PVC)	Persistent state
Packaging	Helm	Templated Kubernetes manifests
Registry	ghcr.io	Container image storage
CI/CD	GitHub Actions	Build, scan, push, deploy
Metrics	Prometheus	Metric collection + alerting
Dashboards	Grafana	Visualisation
Logs	Loki + Promtail	Log aggregation
Security scan	Trivy	Image vulnerability scanning

3. Solution Architecture

The system follows a GitOps-style flow: a push to the main branch triggers CI/CD, which deploys to Kubernetes, which is continuously monitored. The diagram below shows the complete architecture as implemented.

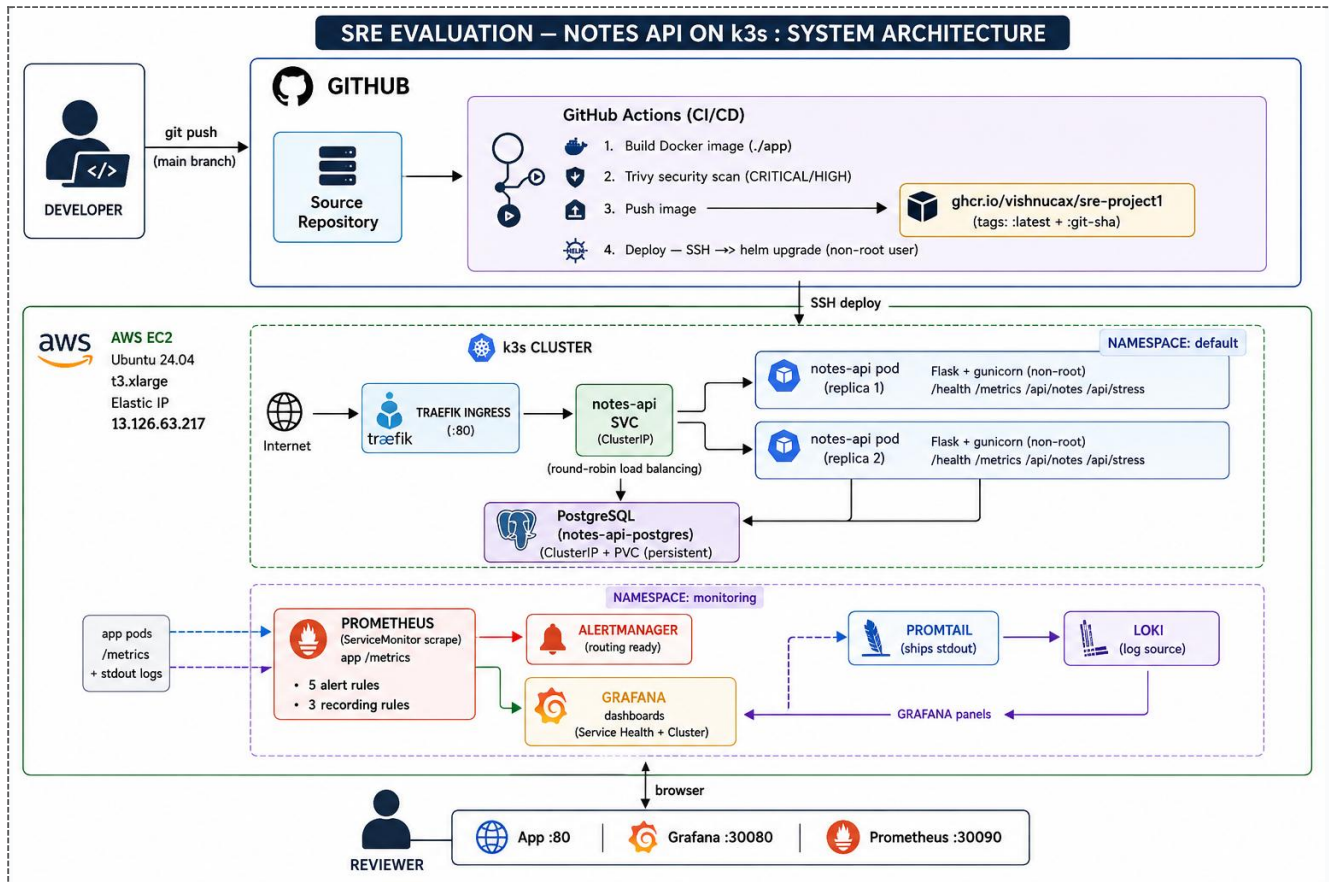


Figure 3.1 — End-to-end system architecture

What to observe: the full path from a developer push through CI/CD to the running, observed service on k3s.

3.1 Component explanations

Component	In simple terms
GitHub Actions	The automation robot: on every push it builds, scans, publishes, and deploys.
ghcr.io	The warehouse where the built container image is stored.
k3s	A small Kubernetes that runs and supervises the containers on one VM.
Traefik ingress	The front door: routes web traffic on port 80 to the app.
notes-api (2 replicas)	Two identical copies of the app for high availability.
PostgreSQL	The database where notes are stored; kept on a persistent volume.
Prometheus	Collects numeric measurements and decides when to raise alarms.
Grafana	The screens: turns measurements and logs into readable dashboards.
Loki + Promtail	Promtail gathers the app's log lines; Loki stores them for search.
Alertmanager	Receives alarms from Prometheus (notification routing is ready to enable).

4. Infrastructure Design

4.1 Compute & operating system

A single AWS EC2 instance running Ubuntu 24.04 LTS (t3.xlarge: 4 vCPU, 16 GB RAM, 40 GB gp3). A permanent Elastic IP (13.126.63.217) keeps the public address stable across stop/start cycles, which also keeps the CI/CD deploy target and the reviewer URLs stable.

4.2 Kubernetes (k3s) & networking

- k3s provides the Kubernetes control plane and runtime in a single lightweight binary.
- Traefik (bundled with k3s) serves as the ingress controller on port 80.
- Two namespaces separate concerns: default (application + database) and monitoring (observability stack).

4.3 Configuration, secrets & storage

Resource	Role
ConfigMap	Non-secret configuration (DB host, port, name, user)
Secret	Database password (never stored in Git)
PersistentVolumeClaim	1 Gi persistent storage for PostgreSQL
ServiceAccount	Dedicated identity with the API token mount disabled (least privilege)
Resource requests/limits	Prevent any single pod from starving the node

Design principle: configuration follows the 12-factor approach — settings come from the environment, and no secret is ever committed to the repository.

5. Application Deployment

5.1 The application

A Flask microservice serving an HTML frontend and a small JSON API. Its endpoints are chosen so each SRE capability can be demonstrated.

Endpoint	Method	Purpose
/	GET	HTML frontend (add / list notes)
/health	GET	Liveness & readiness probe endpoint
/metrics	GET	Prometheus metrics
/api/notes	GET / POST	List / create notes in PostgreSQL
/api/stress	POST	CPU busy-loop (drives the high-CPU scenario)

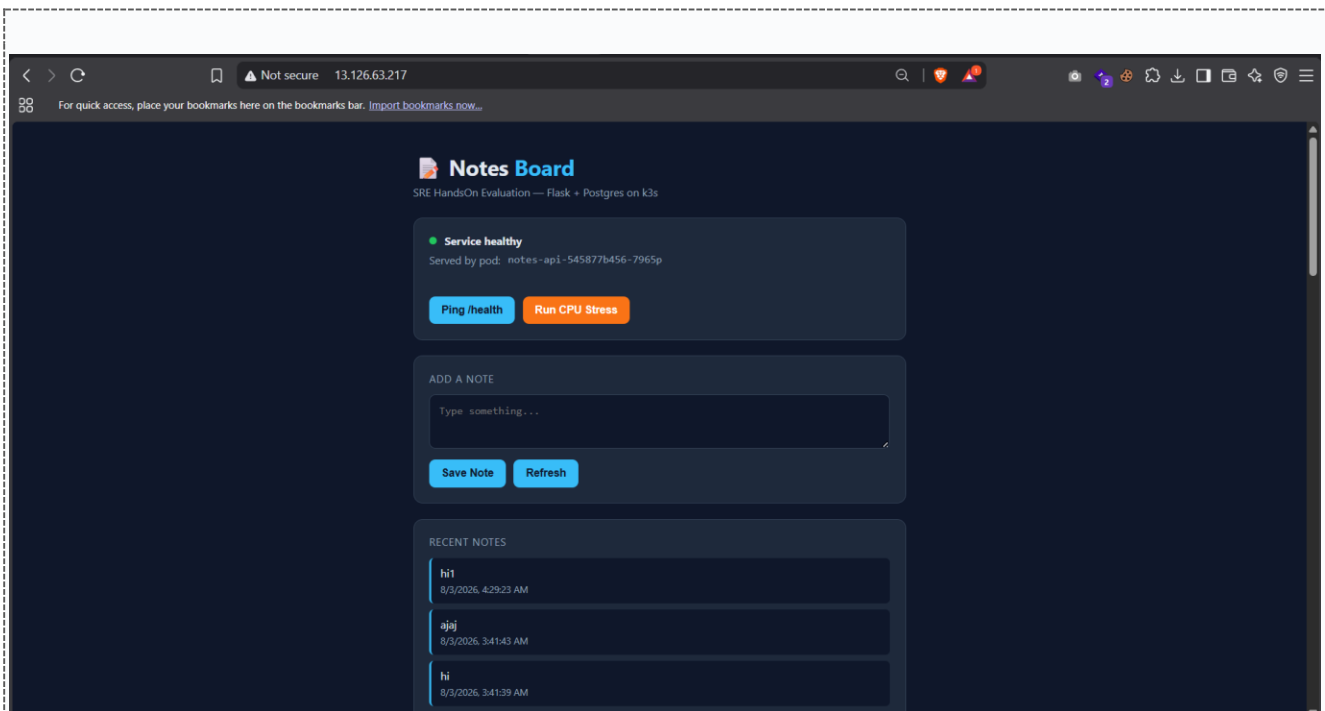


Figure 5.1 — The Notes API frontend

What to observe: the service reports healthy, shows the serving pod, and lists notes persisted in PostgreSQL.

5.2 Container image

- Base image python:3.12-slim for a small footprint and reduced attack surface.
- Runs as a non-root user to limit blast radius if the container is compromised.
- gunicorn as the production WSGI server (not the single-threaded Flask dev server).

5.3 Deployment flow

The Helm chart renders and applies the deployment (2 replicas), service, ingress, ConfigMap, Secret, ServiceAccount, PostgreSQL (with PVC), NetworkPolicy, ServiceMonitor, and PrometheusRule.

Health checks

- Liveness probe on /health — if it fails, Kubernetes restarts the pod (self-healing).
- Readiness probe on /health — gates traffic until the pod is ready to serve.

6. CI/CD Pipeline

GitHub Actions automates the entire lifecycle. Every push to the main branch runs the pipeline below.

Stage	Action	Detail
1. Build	Docker build	Builds the image from ./app
2. Scan	Trivy scan	Reports CRITICAL/HIGH vulnerabilities
3. Push	Publish image	To ghcr.io, tagged :latest and the git commit SHA
4. Deploy	helm upgrade over SSH	As a non-root user with a scoped kubeconfig
5. Verify	Rollout status	Waits for the deployment to become ready

The screenshot displays the GitHub Actions interface for a repository named 'vishnucax / SRE-Project1'. The 'Actions' tab is selected, showing a list of workflow runs under the 'Build and Deploy' workflow. The most recent run, 'ui image added' (Build and Deploy #20), is highlighted. This run was triggered by a push to the 'main' branch and completed successfully with a total duration of 1m 44s. Below the workflow list, the details for the 'ui image added' run are shown. The 'Summary' tab is active, displaying a 'build-deploy.yml' workflow diagram with two jobs: 'build' (52s) and 'deploy' (30s), both of which completed successfully. A 'build summary' section is also visible, providing a 'Docker Build summary' and a link to download the build record archive for import into Docker Desktop's Builds view.

Figure 6.1 — GitHub Actions pipeline (build and deploy successful)

What to observe: both the build and deploy jobs complete successfully (green), proving the pipeline works end to end.

6.1 Key design points

Immutable image tags: every commit produces an image tagged with its git SHA. This gives reproducible, auditable deploys and eliminated a real stale-image caching problem where k3s reused an old :latest image.

Least-privilege deploy: the deploy job connects as a non-root user with a scoped kubeconfig — not host root. If CI credentials were leaked, the blast radius excludes root on the VM.

7. Monitoring & Observability

The observability stack was installed via Helm in a dedicated monitoring namespace: kube-prometheus-stack (Prometheus, Grafana, Alertmanager, node-exporter, kube-state-metrics) and loki-stack (Loki, Promtail).

7.1 Metrics & scraping

Prometheus discovers the application through a ServiceMonitor — the Operator's declarative way of defining a scrape target — and scrapes /metrics every 15 seconds. Cluster and node metrics come from the stack's built-in exporters.

7.2 Alerting rules

Alert	Fires when	Scenario
NotesApiPodCrashLooping	container restart rate > 0 (2m)	Pod crash loop
NotesApiHighCPU	pod CPU > 0.4 cores (2m)	High CPU
NotesApiPodNotReady	pod not ready (5m)	Startup / DB issues
NotesApiDown	up == 0 (1m)	App / DB down
NotesApiHighErrorRate	5xx rate > 0.2 (2m)	DB connectivity loss

7.3 Recording rules

Recording rules pre-compute frequently-used expressions and store them as new series, so dashboards and alerts query the cheaper pre-aggregated metric.

Recording rule	Meaning
notes_api:request_rate:sum	Total request rate across replicas
notes_api:error_rate:ratio	Fraction of requests that are 5xx (an error-ratio SLI)
notes_api:cpu_cores:sum_by_pod	Per-pod CPU in cores (reused by the High CPU alert)

8. Grafana Dashboards

Two dashboards satisfy the observability requirement: a custom service-health dashboard and the cluster/infrastructure dashboards bundled with the monitoring stack.

8.1 Service-health dashboard

A custom 8-panel dashboard, set as the default home dashboard, focused on the application's own signals.

Panel	Why it matters
Request Rate by Status	Shows traffic volume and error mix (spikes in 5xx indicate failure)
Request Latency p95 by Path	The slow-tail latency SLI per endpoint
Total Notes Created	Application activity (per-pod counter)
App Replicas Up	Availability — should read 2 (high availability)
App Memory per Pod	Resource usage; helps spot leaks
Total Notes in DB	Authoritative note count from the database (gauge)
Application Logs	Live log stream from Loki
Application Errors & Warnings	Filtered error logs for fast triage

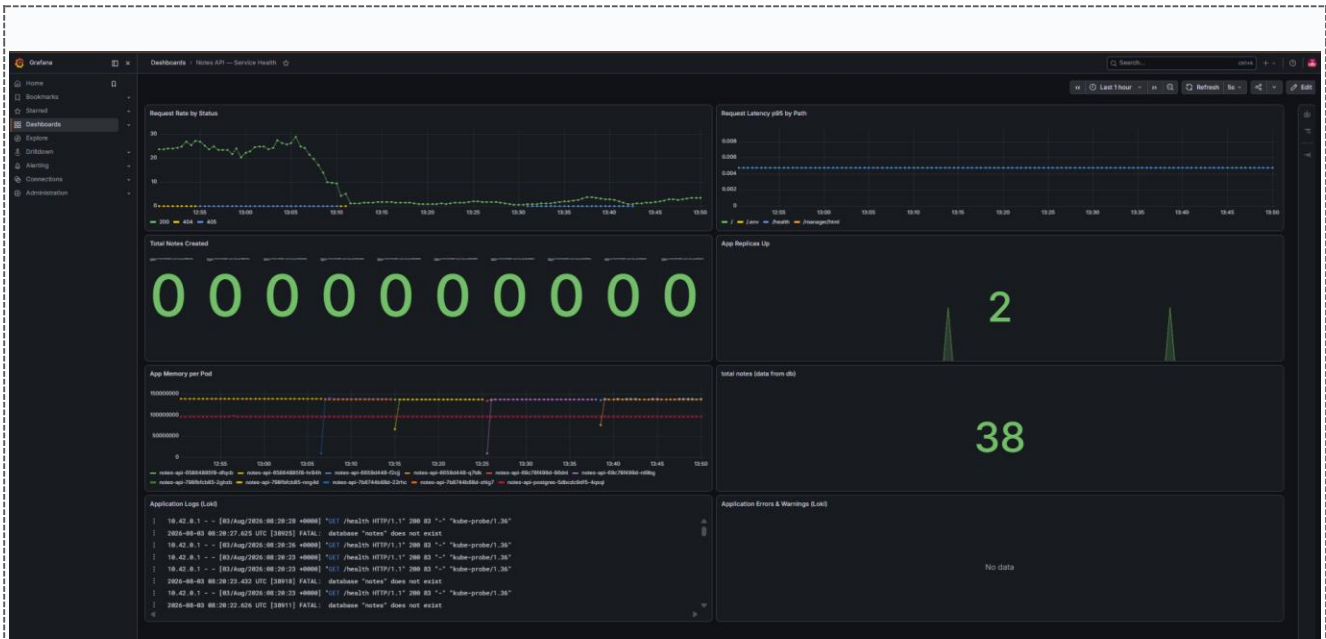


Figure 8.1 — Service-health dashboard (metrics + logs together)

What to observe: metrics and logs on one screen — the request-status panel and the Loki error panel let an operator see what broke and why in one place.

8.2 Cluster / infrastructure dashboard

The Node Exporter and Kubernetes dashboards provide node-level CPU, memory, disk, and cluster object metrics. Both dashboards are exported as JSON in the repository with import instructions.



Figure 8.2 — Cluster / node metrics dashboard(used default template)

What to observe: live CPU, memory, and disk metrics for the underlying VM, confirming infrastructure-level observability.

9. Logging Architecture

Logs flow from the application to Grafana through Loki. The application logs to standard output; Promtail (running on the node) collects those log lines and ships them to Loki, which Grafana queries.

Step	What happens
1. Application	Writes structured logs to stdout (unbuffered)
2. Promtail	Tails pod stdout on the node; labels streams (namespace, pod, container)
3. Loki	Stores and indexes the log streams
4. Grafana	Queries Loki with LogQL (e.g. {namespace="default"})

Why this matters: metrics tell you what changed; logs tell you why. Having both in one Grafana view is the core of fast incident investigation.

10. Failure Simulations

Three scenarios were chosen across different failure classes — configuration, dependency, and resource — to demonstrate breadth. Each was induced with a script, detected by an alert, and recovered. Detailed postmortems appear in Section 11.

10.1 Simulation 1 — Pod Crash Loop (configuration failure)

Objective

Verify detection and resilience when a bad startup configuration causes CrashLoopBackOff.

Steps & command

```
./tests/simulate-pod-crashloop.sh # inject broken startup command
./tests/simulate-pod-crashloop.sh --recover # restore
```

Expected vs actual

Expected: new pods crash-loop; the alert fires; healthy replicas keep serving. Actual: exactly this — the original replicas served throughout (HA held); NotesApiPodCrashLooping fired in ~2m 28s.

Detection & recovery

- Detected by NotesApiPodCrashLooping (also NotesApiDown, NotesApiPodNotReady).
- Recovered by removing the broken command; rollout returned to healthy in ~11 seconds.

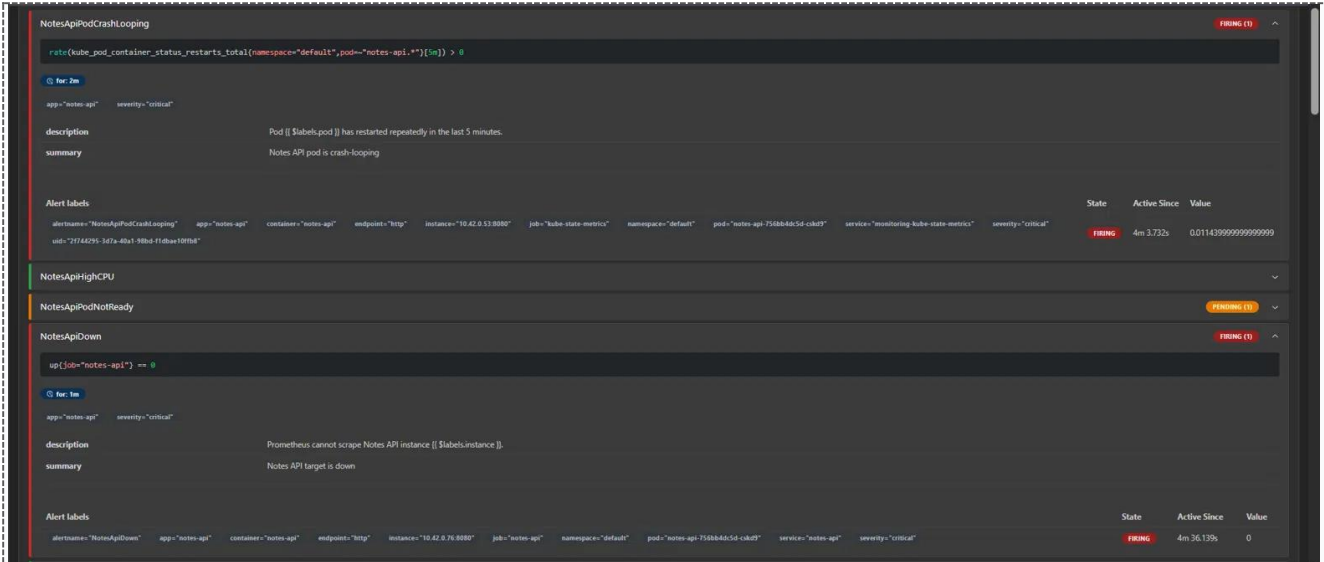


Figure 10.1 — NotesApiPodCrashLooping firing

What to observe: the alert in the firing state, confirming detection of the crash loop.

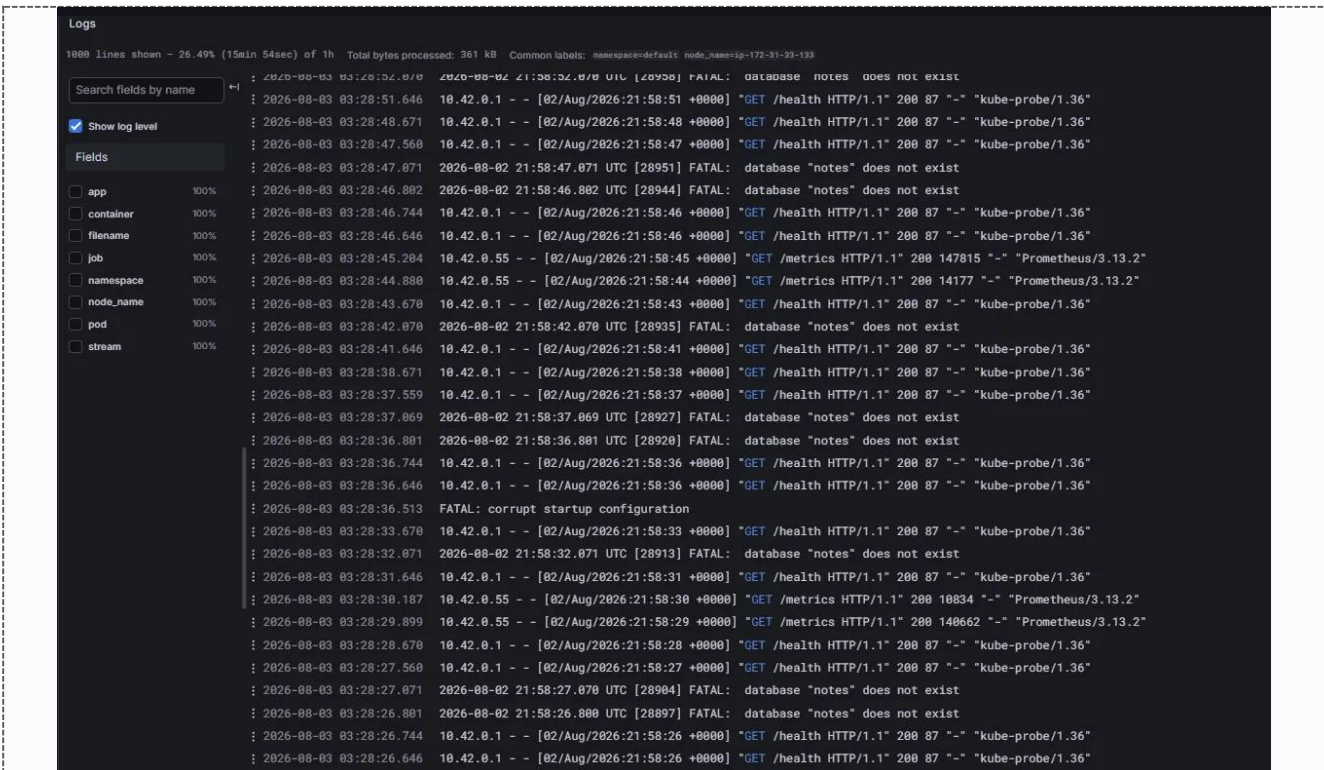


Figure 10.2 — Loki logs during the crash loop

What to observe: the application logs captured the corrupt-startup failure, providing the 'why'.

Lesson learned

High availability protects against pod/node failure, but not against a bad configuration shared by every replica.

10.2 Simulation 2 — Database Connectivity Loss (dependency failure)

Objective

Verify graceful degradation and detection when the database becomes unavailable.

Steps & command

```
./tests/simulate-db-outage.sh # scale Postgres to 0
# sustained load keeps the 5xx rate above threshold:
for i in $(seq 1 480); do curl -s -o /dev/null -w "%{http_code}\n" \
  http://13.126.63.217/api/notes; sleep 0.5; done
./tests/simulate-db-outage.sh --recover # scale Postgres back to 1
```

Expected vs actual

Expected: DB-dependent endpoints return 503; the app stays alive; the alert fires. Actual: exactly this — the app degraded gracefully (503, not a crash), stayed healthy on /health, and NotesApiHighErrorRate fired in ~2m 12s at roughly eight times the threshold.

Detection & recovery

- Detected by NotesApiHighErrorRate; the Loki log 'connection refused' pinpointed the database process being down (versus a timeout, which would indicate a network partition).
- Recovered automatically once the database returned — no app restart needed.

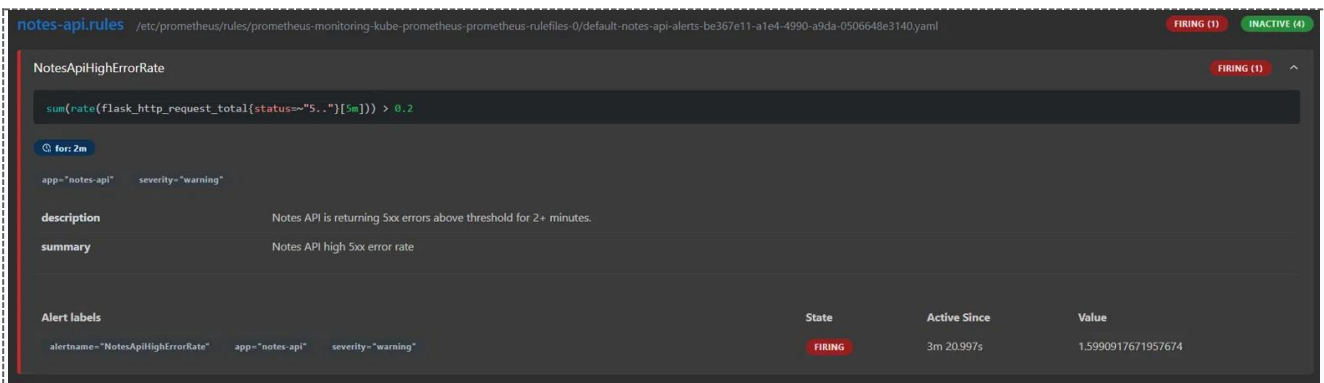


Figure 10.3 — NotesApiHighErrorRate firing

What to observe: the error-rate alert firing during the database outage.

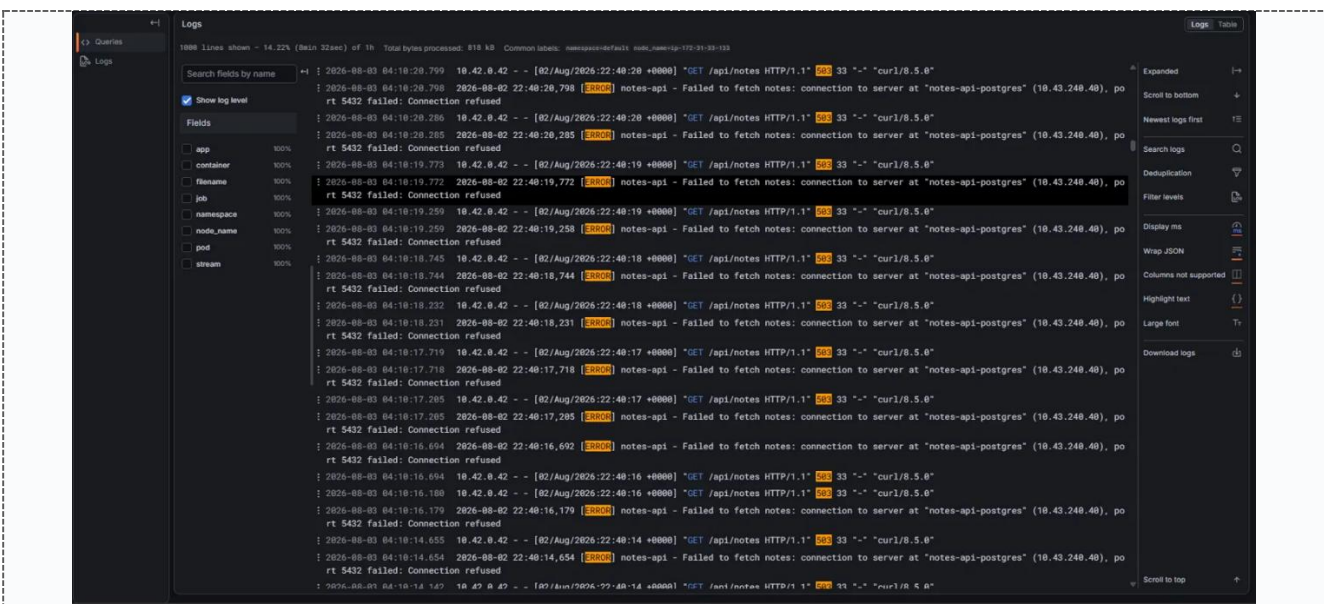


Figure 10.4 — Loki 'connection refused' errors

What to observe: the precise application error, which speeds up triage.

Lesson learned

The in-cluster single-replica database is a single point of failure; the app's graceful 503 degradation is the correct behaviour under a dependency outage.

10.3 Simulation 3 — High CPU Exhaustion (resource failure)

Objective

Verify detection and blast-radius containment when a service is driven to high CPU.

Steps & command

```
./tests/simulate-high-cpu.sh          # sustained parallel /api/stress load
kubectl top pods -l app=notes-api     # observe CPU
```

Expected vs actual

Expected: CPU rises on both replicas; the alert fires; resource limits prevent node starvation. Actual: both pods reached ~0.55 cores (above the 0.4-core threshold, below the 1-core limit); NotesApiHighCPU fired in ~2m 38s; no restart occurred.

Detection & recovery

- Detected by NotesApiHighCPU (fired for both pods).
- Recovered automatically when the load stopped; the alert cleared after the metric averaging window aged out the spike.

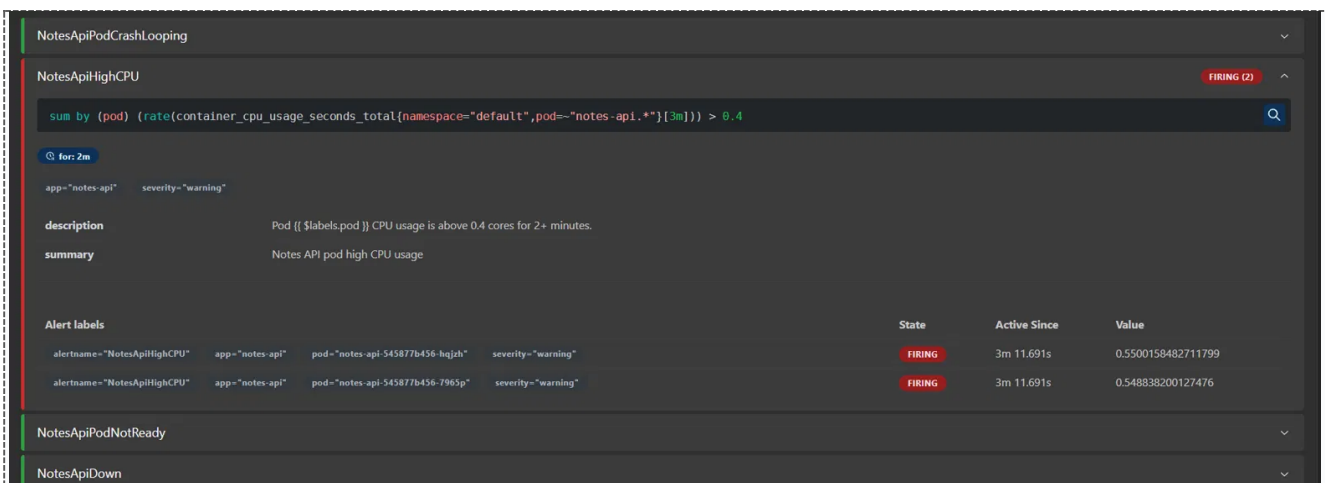


Figure 10.5 — NotesApiHighCPU firing

What to observe: the CPU alert firing for both replicas at ~0.55 cores.

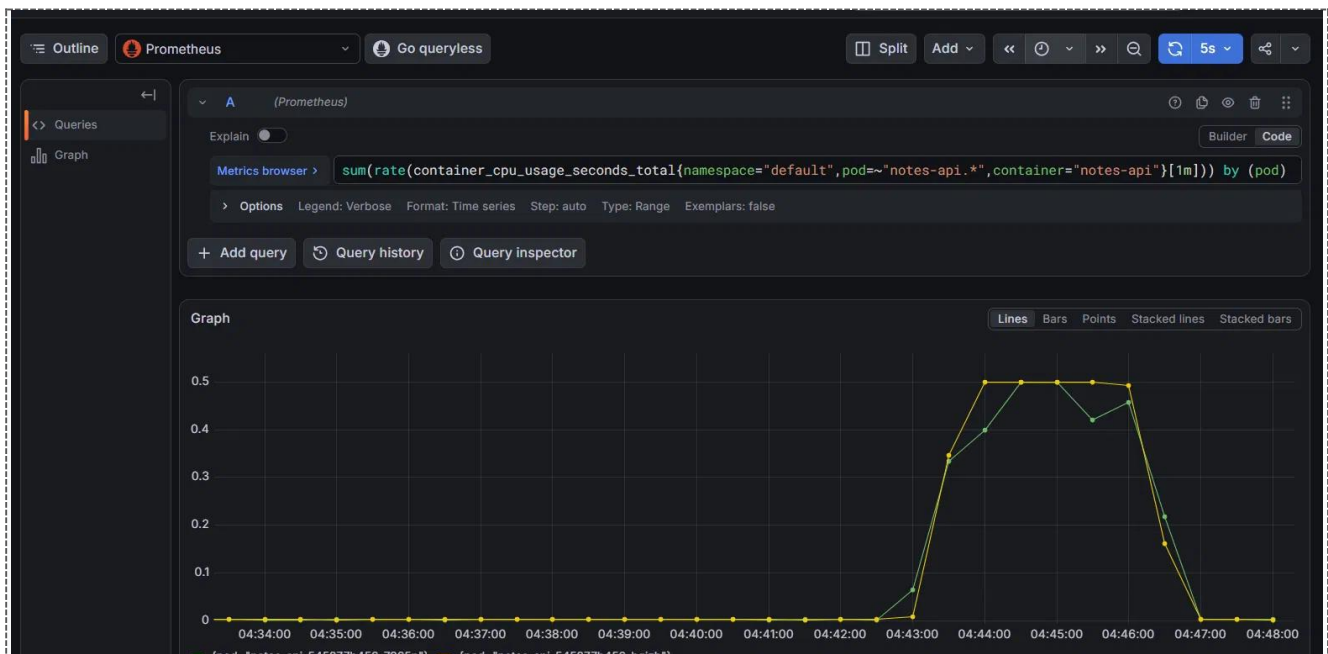


Figure 10.6 — Grafana CPU spike

What to observe: the CPU rise and fall across both pods during the incident.

Lesson learned

Kubernetes CPU limits act as a blast-radius ceiling; an unthrottled compute-bound endpoint is a denial-of-service vector worth rate-limiting in production.

11. Incident Response (Postmortems)

Each incident is documented as a blameless postmortem using the five-whys method. The full documents are in the repository (docs/rca-01/02/03). Summaries follow.

11.1 Incident #1 — Pod Crash Loop

Field	Value
Severity	SEV-3 (degraded capacity; no user outage due to HA)
Primary alert	NotesApiPodCrashLooping
Time-to-detection	~2m 28s
Status	Resolved

Five whys → root cause

- The container exited immediately with a non-zero status.
- Its startup command was overridden with a malformed sequence.
- Kubernetes restarts failed containers with exponential backoff → CrashLoopBackOff.
- The bad command reached the cluster via a direct change (simulating a bad config/entrypoint).
- ROOT CAUSE: configuration errors are shared by all identical replicas, so HA does not protect against them.

Corrective actions

Action	Owner	Priority
Document crash-loop response in the runbook	Vishnu K	High

Add a CI smoke test hitting /health before deploy	Vishnu K	Medium
Adopt helm --atomic for auto-rollback on failed deploys	Vishnu K	Medium

Incidental finding: log review surfaced an unrelated, harmless line — FATAL: database "notes" does not exist — caused by the Postgres readiness probe defaulting to a database named after the user. Fix (low priority): pg_isready -U notes -d notesdb.

11.2 Incident #2 — Database Connectivity Loss

Field	Value
Severity	SEV-2 (major functionality unavailable; no data loss)
Primary alert	NotesApiHighErrorRate
Time-to-detection	~2m 12s
Status	Resolved

Five whys → root cause

- Users received HTTP 503 on /api/notes.
- The app could not connect to PostgreSQL (connection refused on port 5432).
- The Postgres pod was not running.
- Postgres runs as a single replica with no standby.
- ROOT CAUSE: the database is a single point of failure with no HA/failover.

Corrective actions

Action	Owner	Priority
Document DB-recovery in the runbook	Vishnu K	High
Add a cause-based alert (NotesApiDatabaseDown)	Vishnu K	Medium
Add a DB-aware readiness gate	Vishnu K	Medium
Production: managed/replicated PostgreSQL with failover + PITR	Platform	High

Analysis highlight: NotesApiHighErrorRate is a symptom-based alert (users seeing 5xx) — broad coverage. A cause-based alert (Postgres replicas == 0) would be faster and more precise. A mature setup uses both.

11.3 Incident #3 — High CPU Exhaustion

Field	Value
Severity	SEV-3 (elevated resource usage; no outage)
Primary alert	NotesApiHighCPU
Time-to-detection	~2m 38s
Status	Resolved

Five whys → root cause

- The pods executed a CPU-intensive busy-loop repeatedly.
- The /api/stress endpoint was called continuously in parallel.
- The load balancer round-robinbed the requests evenly across both replicas.
- The endpoint has no rate-limiting or concurrency cap.
- ROOT CAUSE: no rate-limiting on a compute-bound endpoint (a DoS vector).

Corrective actions

Action	Owner	Priority
Document high-CPU / scaling response in the runbook	Vishnu K	High
Add rate-limiting / concurrency caps (or gate /api/stress)	Vishnu K	Medium
Add a Horizontal Pod Autoscaler (HPA) on CPU	Vishnu K	Medium

Analysis highlight: the 1-core CPU limit per pod contained the blast radius; the alert's ~3-minute auto-resolution lag is expected because the rate window must age out the spike.

12. Runbooks

The operational runbook (docs/runbook.md) opens with triage and routes to the relevant remediation. Key procedures are summarised below.

12.1 First five minutes (triage)

```
kubectl get pods -A | grep -Ev "Running|Completed"
curl -s -o /dev/null -w "HTTP %{http_code}\n" http://13.126.63.217/health
# review firing alerts at http://13.126.63.217:30090/alerts
```

12.2 Pod crash loop

Symptoms: pods restarting; NotesApiPodCrashLooping firing. Diagnosis: inspect the crashing pod's logs. Remediation: roll back a bad release.

```
kubectl logs <POD> --previous --tail=50
helm history notes-app
helm rollback notes-app <PREVIOUS_REVISION>
```

12.3 Database connectivity loss

Symptoms: 503 on /api/notes; 'connection refused' in logs. Remediation: restore Postgres.

```
kubectl scale deployment notes-api-postgres --replicas=1
curl -s http://13.126.63.217/api/notes | head -c 200 # verify
```

12.4 High CPU

Symptoms: NotesApiHighCPU firing; elevated latency. Remediation: scale out, identify the hot endpoint.

```
kubectl top pods -l app=notes-api
kubectl scale deployment notes-api --replicas=4
```

13. Security

Control	Status
No secrets in repo (ConfigMap/Secret + GitHub Secrets)	Implemented
RBAC least privilege (dedicated SA, API token disabled)	Implemented
CI/CD deploys as non-root (scoped kubeconfig)	Implemented
Non-root container, minimal python:3.12-slim base	Implemented
Image vulnerability scanning (Trivy CRITICAL/HIGH)	Implemented
Network segmentation (NetworkPolicy)	Declared; not enforced on k3s Flannel
Secrets encryption-at-rest	Out of scope (production: KMS/Vault)
Database high availability	Out of scope (single-node PoC)

13.1 Secrets handling

Configuration follows the 12-factor approach. Non-secret values live in a ConfigMap; the database password lives in a Kubernetes Secret; CI credentials (VM host, user, SSH key) live in GitHub Secrets. No secret is committed to the repository, and the Helm values carry only a placeholder password overridden at deploy time.

13.2 Network segmentation caveat

Honest limitation: a NetworkPolicy restricting Postgres ingress to the app is declared, but k3s's default Flannel CNI does not enforce NetworkPolicies. The policy documents correct intent; enforcement in production requires a policy-capable CNI such as Calico.

13.3 Observed exploit scanning

Within hours of public exposure, the /metrics endpoint recorded automated bot traffic probing paths such as /vendor/phpunit/.../eval-stdin.php — all safely rejected (the app is Python, not PHP). This reinforces the value of a minimal attack surface and non-root containers.

14. Project Folder Structure

Path	Contents
app/	Flask app, Dockerfile, HTML frontend
helm/notes-app/	Helm chart (13 templates)
helm/notes-app/dashboards/	dashboard.json (service health) + dashboard-cluster.json
.github/workflows/build-deploy.yml	CI/CD pipeline
tests/	Three failure-simulation scripts
docs/rca-01/02/03-*.md	Root-cause analyses (one per incident)
docs/runbook.md	Operational runbook
docs/security.md	Security notes & secrets handling
docs/design-notes.md	Key decisions & trade-offs
docs/prometheus-config-notes.md	Scrape / alerting / recording rules
docs/loki-stack-values.yaml	Loki + Promtail configuration
docs/grafana-dashboards-README.md	Dashboard import instructions
docs/ALL-COMMANDS.txt	Full command reference
docs/evidence/	Screenshots + architecture diagram

15. Challenges Faced

Challenge	Resolution
unicorn didn't create the DB table	init_db() ran only under __main__; moved to import scope. Surfaced by reading pod logs.
k3s reused a stale :latest image	Switched to immutable git-SHA image tags in CI/CD.
CI deploy failed on kubeconfig permissions	Used a scoped, user-owned kubeconfig and deployed as non-root.
Loki appeared unreachable in Grafana	The data-source URL needed the correct in-cluster service name (http://loki:3100).
Per-pod metrics looked uneven	Realised port-forward pins to one pod; testing via the ingress showed correct round-robin.
k3s control-plane alerts firing	Identified as expected false positives due to k3s's bundled topology.

16. Lessons Learned

- High availability and self-healing are different guarantees; neither protects against a bad shared configuration.
- How you test determines what you observe — a debugging shortcut (port-forward) can hide the true behaviour.
- The right metric type matters: counters for rates, gauges for current state.
- Detection latency is a deliberate trade-off between speed and alert noise.
- Honestly documenting a limitation is more valuable than hiding it.

17. Future Improvements

- Multi-node cluster and managed/replicated PostgreSQL with backups to remove single points of failure.

- A policy-capable CNI (Calico) so NetworkPolicies are enforced.
- Alertmanager notification routing (email/Slack/PagerDuty) for on-call delivery.
- A Horizontal Pod Autoscaler and rate-limiting on compute-bound endpoints.
- Secrets manager (Vault / External Secrets) and encryption-at-rest for Kubernetes Secrets.

18. Conclusion

This project delivers a small but production-shaped service that is deployed automatically, observed continuously, and proven to be operable under failure. The emphasis throughout has been on sound engineering decisions, honest documentation of limitations, and demonstrable incident response — the substance of Site Reliability Engineering. All artefacts are in the repository, the environment is live for review, and the remaining production hardening is clearly identified for the next phase.

Appendix A — Key Commands

```
# Deploy / upgrade the app
helm upgrade --install notes-app ./helm/notes-app --wait

# Observe
kubectl get pods -l app=notes-api
kubectl top pods -l app=notes-api
kubectl logs -f -l app=notes-api

# Recover
kubectl rollout restart deployment notes-api
helm rollback notes-app <revision>
kubectl scale deployment notes-api-postgres --replicas=1
```

Full reference: docs/ALL-COMMANDS.txt

Appendix B — Configuration Files

File	Purpose
helm/notes-app/values.yaml	Chart configuration
helm/notes-app/templates/prometheusrule.yaml	Alert + recording rules
helm/notes-app/templates/servicemonitor.yaml	Prometheus scrape config
docs/loki-stack-values.yaml	Loki + Promtail configuration
.github/workflows/build-deploy.yml	CI/CD pipeline

Appendix C — References

- k3s — <https://k3s.io>
- kube-prometheus-stack & loki-stack Helm charts
- Prometheus, Grafana, Loki, Trivy official documentation

Appendix D — Abbreviations

Term	Meaning
SRE	Site Reliability Engineering

CI/CD	Continuous Integration / Continuous Deployment
RCA	Root-Cause Analysis
HA	High Availability
PVC	PersistentVolumeClaim
SLI	Service Level Indicator
CNI	Container Network Interface
SPOF	Single Point of Failure
TTD	Time-to-Detection
CVE	Common Vulnerabilities and Exposures

Appendix E — Glossary

- CrashLoopBackOff — a state where Kubernetes repeatedly restarts a failing container with increasing delays.
- Recording rule — a pre-computed Prometheus expression stored as a new metric for cheaper reuse.
- ServiceMonitor — a custom resource that declaratively tells Prometheus what to scrape.
- Graceful degradation — returning a controlled error (503) instead of crashing when a dependency fails.
- Blast radius — the extent of damage a failure or compromise can cause.